

Chapter 6

Introduction to Structured Query Language (SQL)

Database Systems:
Design, Implementation, and
Management, Sixth Edition, Rob and
Coronel

Introduction to SQL

- ❑ SQL functions fit into two broad categories:
- ❑ Data definition language
 - SQL includes commands to create
 - ❑ Database objects such as tables, indexes, and views
 - ❑ Commands to define access rights to those database objects
- ❑ Data manipulation language
 - Includes commands to insert, update, delete, and retrieve data within the database tables

Introduction to SQL

- ❑ SQL is relatively easy to learn
- ❑ Basic command set has a vocabulary of less than 100 words
- ❑ Nonprocedural language
- ❑ American National Standards Institute (ANSI) prescribes a standard SQL
- ❑ Several SQL dialects exist

SQL Data Definition Commands

TABLE 6.1 SQL DATA DEFINITION COMMANDS

COMMAND OR OPTION	DESCRIPTION
CREATE SCHEMA AUTHORIZATION	Creates a database schema
CREATE TABLE	Creates a new table in the user's database schema
NOT NULL	Constraint that ensures that a column will not have null values
UNIQUE	Constraint that ensures that a column will not have duplicate values
PRIMARY KEY	Defines a primary key for a table
FOREIGN KEY	Defines a foreign key for a table
DEFAULT	Defines a default value for a column (when no value is given)
CHECK	Constraint used to validate data in a column
CREATE INDEX	Creates an index for a table
CREATE VIEW	Creates a dynamic subset of rows/columns from one or more tables
ALTER TABLE	Modifies a table's definition (adds, modifies, or deletes attributes or constraints)
CREATE TABLE AS	Creates a new table based on a query in the user's database schema
DROP TABLE	Permanently deletes a table (and thus its data)
DROP INDEX	Permanently deletes an index
DROP VIEW	Permanently deletes a view

Data Manipulation Commands

TABLE 6.2 DATA MANIPULATION COMMANDS

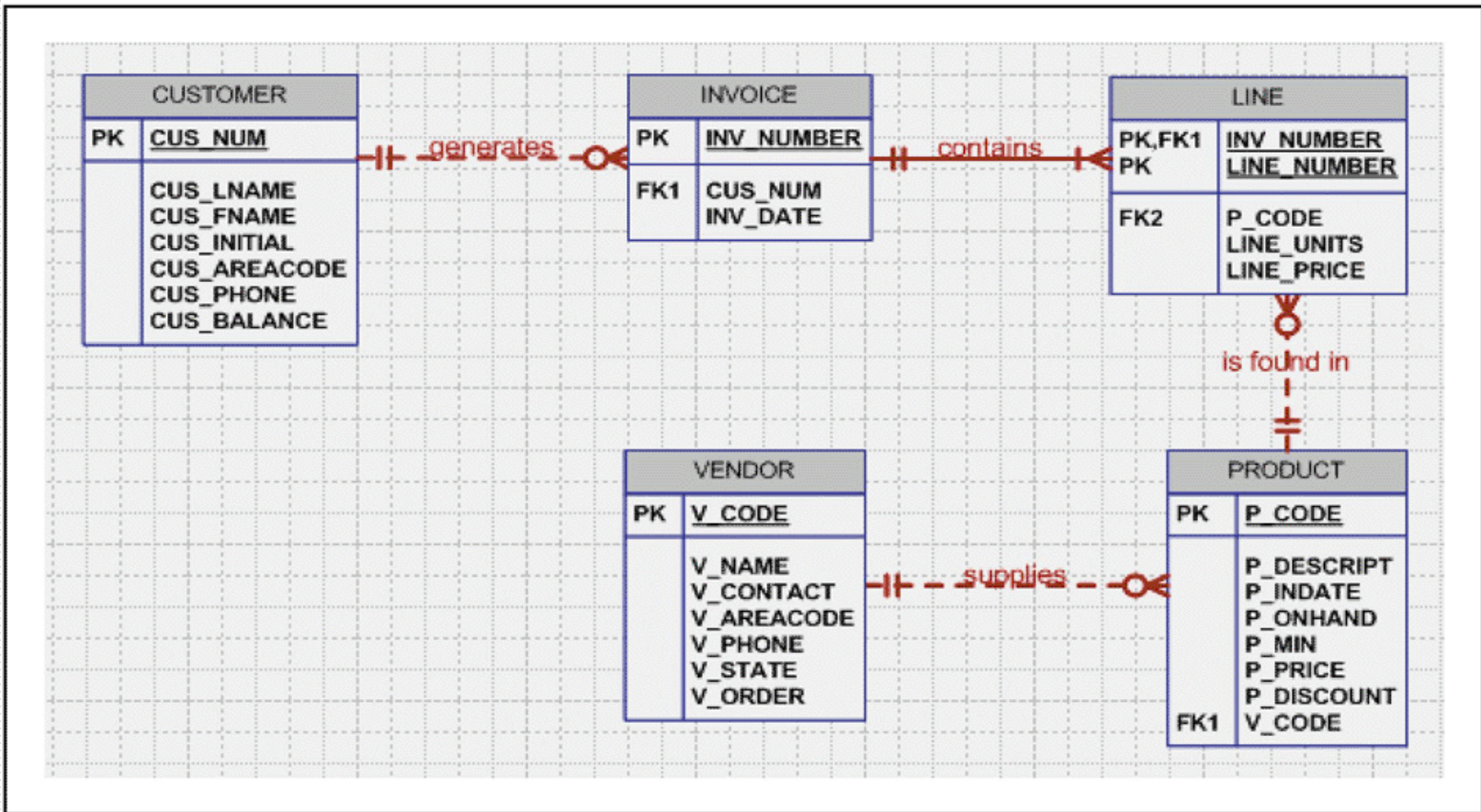
COMMAND OR OPTION	DESCRIPTION
INSERT	Inserts row(s) into a table
SELECT	Selects attributes from rows in one or more tables or views
WHERE	Restricts the selection of rows based on a conditional expression
GROUP BY	Groups the selected rows based on one or more attributes
HAVING	Restricts the selection of grouped rows based on a condition
ORDER BY	Orders the selected rows

Data Definition Commands

- ❑ Examine the simple database model and the database tables that will form the basis for the many SQL examples
- ❑ Understand the data environment

The Database Model

FIGURE 6.1 THE DATABASE MODEL



The Database Model

- ❑ Reflects the following business rules
 - A customer may generate one or more invoices; each invoice is generated by one customer
 - An invoice contains one or more invoice lines; each invoice line is associated with one invoice
 - Each invoice line references one product; a product may be found in many invoice lines
 - A vendor *may* supply many products; some vendors do not yet supply products
 - If a product is vendor-supplied, that product is supplied by only a single vendor
 - Some products are not supplied by a vendor (in-house or bought on the open market)

P_CODE	P_DESCRIPTOR	P_INDATE	P_ONHAND	P_MIN	P_PRICE	P_DISCOUNT	V_CODE				
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-03	8	5	\$109.99	0.00	25595				
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-03	32	15	\$14.99	0.05	21344				
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-03	18	12	\$17.49	0.00	21344				
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-04	15	8	\$39.95	0.00	23119				
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-04	23	5	\$43.99	0.00	23119				
2232/PTY	B&D jigsaw, 12-in. blade	30-Dec-03	8	5	\$109.92	0.05	24288				
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-03	6	5	\$99.87	0.05	24288				
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-04	12	5	\$38.95	0.05	25595				
23109-HB	Claw hammer	20-Jan-04	23	10	\$9.95	0.10	21225				
23114-AA	Sledge hammer, 12 lb.	02-Jan-04	8	5	\$14.40	0.05					
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-03	43	20	\$4.99	0.00	21344				
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-04	11	5	\$256.99	0.05	24288				
PVC23DR T	PVC pipe, 3.5-in., 8-ft	20-Feb-04	188	75	\$5.87	0.00					
SM-18277	1.25-in. metal screw, 25	01-Mar-04	172	75	\$6.99	0.00	21225				
SW-23116	2.5-in. wd. screw, 50	24-Feb-04	237	100	\$8.45	0.00	21231				
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-04	18	5	\$119.95	0.10	25595	V_CODE	V_PHONE	V_STATE	V_ORDER
			21225	Bryson, Inc.		Smithson	615	223-3234	TN	Y	
			21226	SuperLoo, Inc.		Flushing	904	215-8995	FL	N	
			21231	D&E Supply		Singh	615	228-3245	TN	Y	
			21344	Gomez Bros.		Ortega	615	889-2546	KY	N	
			22567	Dome Supply		Smith	901	678-1419	GA	N	
			23119	Randsets Ltd.		Anderson	901	678-3998	GA	Y	
			24004	Brackman Bros.		Browning	615	228-1410	TN	N	
			24288	ORDVA, Inc.		Hakford	615	898-1234	TN	Y	
			25443	B&K, Inc.		Smith	904	227-0093	FL	N	
			25501	Damal Supplies		Smythe	615	890-3529	TN	N	
			25595	Rubicon Systems		Orton	904	456-0092	FL	Y	

The Database Model

- ❑ Note the following regarding VENDOR and PRODUCTS
 - The VENDOR table contains vendors who are not referenced in the PRODUCTS table
 - ❑ PRODUCT is optional to VENDOR
 - Existing V_CODE values in the PRODUCT table must have a match in the VENDOR table to ensure referential integrity
 - A few products are supplied factory-direct, a few are made in-house and a few have been bought in a special warehouse sale. Thus, a product is not necessarily supplied by a vendor – VENDOR is optional to PRODUCT

Creating the Database

- ❑ Two tasks must be completed
 - create the database structure
 - create the tables that will hold the end-user data
- ❑ First task
 - RDBMS creates the physical files that will hold the database
 - Tends to differ substantially from one RDBMS to another

The Database Schema

❑ Authentication

- Process through which the DBMS verifies that only registered users are able to access the database
- Log on to the RDBMS using a user ID and a password created by the database administrator

❑ Schema

- Logical grouping of database objects—such as tables and indexes—that are related to each other

Data Types

- ❑ Data type selection is usually dictated by the nature of the data and by the intended use
- ❑ Pay close attention to the expected use of attributes for sorting and data retrieval purposes
 - A field that is all numeric, but upon which no mathematical computation will be performed, should be stored as character for faster query processing.
 - However, if you sort on a number stored as character, 10 will come before 2 because of the ANSI character comparison (e.g., number of bedrooms)

Some Common SQL Data Types

TABLE 6.4 SOME COMMON SQL DATA TYPES

DATA TYPE	FORMAT	COMMENTS
Numeric	NUMBER(L,D)	Used to store numbers (fixed and floating point) in the range 10^{-130} to 10^{126} , with up to 38 decimal places. Allows you to specify numbers in terms of storage length and the number of decimal places. The declaration NUMBER(7,2) indicates numbers that will be stored with two decimal places and may be up to six digits long, including the sign and the decimal place. Examples: 12.32, -134.99. If you specify only a single parameter by writing NUMBER(7), only the number length is specified, allowing you to store numbers that are up to six digits long, without specifying the number of decimal places. Therefore, both 0.023 and -243562 are acceptable when you specify NUMBER(7). The designation NUMBER, without parameters, is also allowed. However, this “no parameter” designation uses the system default specifications for any numbers you may store.
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places. Up to 11 digits. Examples: 12, 36576, 9989765. Like INTEGER, but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT to produce better storage optimization. Like the NUMBER specification, but the storage length is a <i>minimum</i> specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable.
	SMALLINT	
Character	DECIMAL(L,D)	
	CHAR(L)	Fixed length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are filled with blanks. Therefore, if you specify CHAR(25), strings like “Smith” and “Katzenjammer” are stored as Smithbbbbbbbbbbbbbbbbbbbb and Katzenjammerbbbbbbbbbbbbbb. (The letter “b” indicates a blank, not the letter “b”). In short, if you must store variable-length strings, do not use CHAR, because this designation wastes storage space. However, a U. S. phone area code is always three digits long, so CHAR(3) would be appropriate if you want to store such codes.
	VARCHAR(L)	Variable-length character data. The designation VARCHAR(25) will let you store characters that are up to 25 characters long. Unlike the CHAR designation, VARCHAR will not store trailing blanks. Therefore, if you specify VARCHAR(25), strings like “Smith” and “Katzenjammer” are stored without trailing blanks. VARCHAR is more efficient than CHAR if variable-length character data are to be stored. Oracle users may use VARCHAR2, as well as VARCHAR.
Date	DATE	Stores dates in the Julian date format. The year value is four digits long, 0001 through 9999, the month values require two digits 01 through 12, and the day values require two digits, 01 through 31.

Creating Table Structures

```
CREATE TABLE VENDOR (  
  V_CODE          INTEGER          NOT NULL UNIQUE,  
  V_NAME          VARCHAR(35)      NOT NULL,  
  V_CONTACT       VARCHAR(15)      NOT NULL,  
  V_AREACODE      CHAR(3)          NOT NULL,  
  V_PHONE         CHAR(8)          NOT NULL,  
  V_STATE        CHAR(2)          NOT NULL,  
  V_ORDER         CHAR(1)          NOT NULL,  
  PRIMARY KEY (V_CODE) );
```

```
CREATE TABLE PRODUCT (  
  P_CODE          VARCHAR(10) NOT NULL UNIQUE,  
  P_DESCRIPT      VARCHAR2(35) NOT NULL,  
  P_INDATE       DATE NOT NULL,  
  P_ONHAND       SMALLINT NOT NULL,  
  P_MIN          SMALLINT NOT NULL,  
  P_PRICE        NUMBER(8,2) NOT NULL,  
  P_DISCOUNT    NUMBER(4,2) NOT NULL,  
  V_CODE         INTEGER,  
  PRIMARY KEY (P_CODE) ,  
  FOREIGN KEY (V_CODE) REFERENCES VENDOR  
  ON UPDATE (CASCADE) );
```

Creating Table Structures

- ❑ Use one line per column (attribute) definition
- ❑ Use spaces to line up the attribute characteristics and constraints
- ❑ Table and attribute names are capitalized
- ❑ NOT NULL specification
- ❑ UNIQUE specification
- ❑ Primary key attributes contain both a NOT NULL and a UNIQUE specification
- ❑ RDBMS will automatically enforce referential integrity for foreign keys
- ❑ Command sequence ends with a semicolon

Other SQL Constraints

- ❑ NOT NULL constraint
 - Ensures that a column does not accept nulls
- ❑ UNIQUE constraint
 - Ensures that all values in a column are unique
- ❑ DEFAULT constraint
 - Assigns a value to an attribute when a new row is added to a table
- ❑ CHECK constraint
 - Validates data when an attribute value is entered
- ❑ ON UPDATE CASCADE
 - Ensures that a change any VENDOR's V_CODE will automatically be applied to all FK references throughout the system
 - Also have ON DELETE CASCADE and ON UPDATE CASCADE

SQL Constraint Examples

❑ **CUS_AREACODE CHAR(3) DEFAULT '615' NOT NULL
CHECK (CUS_AREACODE IN ('615','713','931')) ,**

- DEFAULT value applies only when new rows are added to a table and only if no value is entered for that field
- CHECK is always validated when a row is added or modified

❑ **CUS_BALANCE NUMBER(9,2) DEFAULT 0.00 ,**

❑ **CONSTRAINT CUS_UI1 UNIQUE (CUS_LNAME, CUS_FNAME)**

SQL Constraint Examples

```
CREATE TABLE INVOICE (  
    INV_NUMBER NUMBER PRIMARY KEY,  
    CUS_CODE NUMBER NOT NULL  
                        REFERENCES CUSTOMER (CUS_CODE) ,  
    INV_DATE DATE DEFAULT SYSDATE NOT NULL,  
    CONSTRAINT INV_CK1  
        CHECK (INV_DATE >  
              TO_DATE (' 01-JAN-2002' , ' DD-MON-YYYY' ) ) ) ;
```

SQL Constraint Examples

```
CREATE TABLE LINE (  
  INV_NUMBER    NUMBER NOT NULL,  
  LINE_NUMBER  NUMBER(2,0) NOT NULL,  
  P_CODE        VARCHAR(10) NOT NULL,  
  LINE_UNITS   NUMBER(9,2) DEFAULT 0.00 NOT NULL,  
  LINE_PRICE   NUMBER(9,2) DEFAULT 0.00 NOT NULL,  
  PRIMARY KEY (INV_NUMBER, LINE_NUMBER),  
  FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE ON DELETE  
  CASCADE,  
  FOREIGN KEY (P_CODE) REFERENCES PRODUCT (P_CODE) ,  
  CONSTRAINT LINE_UI1 UNIQUE (INV_NUMBER, P_CODE) ) ;
```

❑ ON DELETE CASCADE recommended for weak entities to ensure that the deletion of a row in the strong entity automatically triggers the deletion of the corresponding rows in the dependent weak entity
DEFAULT value applies only when new rows are added to a table and only if no value is entered for that field

- Deletion of an INVOICE row will automatically delete all the LINE rows related to that invoice

SQL Indexes

- ❑ When a primary key is declared, DBMS automatically creates a unique index
- ❑ Often need additional indexes
- ❑ Using the CREATE INDEX command, SQL indexes can be created on the basis of any selected attribute
 - `CREATE [UNIQUE] INDEX P_CODEX ON PRODUCT(P_CODE);`
- ❑ Composite index
 - Index based on two or more attributes, ordered left to right
 - ❑ `KEY(INV_NUMBER, LINE_NUMBER)`
 - Often used to prevent data duplication

A Duplicated TEST Record

TABLE 6.5 A DUPLICATED TEST RECORD

EMP_NUM	TEST_NUM	TEST_CODE	TEST_DATE	TEST_SCORE
110	1	WEA	15-May-2003	93
110	2	WEA	12-May-2004	87
111	1	HAZ	14-Dec-2002	91
111	2	WEA	18-Feb-2004	95
111	3	WEA	18-Feb-2004	95
112	1	CHEM	17-Aug-2003	91

```
CREATE UNIQUE INDEX EMP_TESTDEX ON  
TEST(EMP_NUM,TEST_CODE,TEST_DATE);
```

This will allow the use of a candidate key as a unique index

SQL Indexes

- ❑ The default ordering for an index is ascending. It can be changed to descending by adding DESC after the field name
 - `CREATE INDEX PROD_PRICEX ON PRODUCT(P_PRICE DESC);`
- ❑ Delete an index with `DROP INDEX indexname;`

Data Manipulation Commands

- ☐ Adding table rows
- ☐ Saving table changes
- ☐ Listing table rows
- ☐ Updating table rows
- ☐ Restoring table contents
- ☐ Deleting table rows
- ☐ Inserting table rows with a select subquery

Data Manipulation Commands

- ❑ INSERT INTO *tablename* VALUES(*value1*, *value2*, ... *valueN*)
 - String and date values must be entered between apostrophes
 - Numerical entries are not enclosed in apostrophes
 - Attribute entries are separated by commas
 - A value is required for each column in the table
 - Fields that accept NULL values can be set through INSERT
 - You can specify the field names and then just supply those values
 - ❑ INSERT INTO PRODUCT(P_CODE, P_DESCRIPT) VALUES ('BRT-45', 'Titanium drill bit');

Common SQL Data Manipulation Commands

TABLE 6.6 COMMON SQL DATA MANIPULATION COMMANDS

COMMAND	DESCRIPTION
INSERT	Lets you insert data into a table, one row at a time. Used to make the initial data entries into a new table structure or to add data to a table that already contains data.
SELECT	Lists the table contents.
COMMIT	Lets you permanently save your work to disk.
UPDATE	Enables you to make changes to column values in one or more data rows.
ROLLBACK	Restores the database table contents to their original condition (since the last COMMIT).
DELETE	Enables you to delete one or more data rows.

A Data View and Entry Form

FIGURE 6.3 A DATA VIEW AND ENTRY FORM

PRODUCT Table Data View and Data Entry

Product code:	11QER/31
Description:	Power painter, 15 psi., 3-nozzle
Stock date::	03-Dec-03
Units on hand:	8
Minimum units:	5
Price:	\$109.99
Discount rate:	0.00
Vendor code:	25595

Duck Data Entry System

Close the product form

Record: 1 of 16
Product code: Primary key

Saving Table Changes

- ❑ Changes made to table contents are not physically saved on disk until
 - Database is closed
 - Program is closed
 - COMMIT command is used
- ❑ Syntax
 - COMMIT [WORK]
- ❑ Will permanently save any changes made to any table in the database

Listing Table Rows

- ❑ SELECT
 - Used to list contents of table
- ❑ Syntax
 - SELECT *columnlist*
FROM *tablename*
- ❑ *columnlist* represents one or more attributes, separated by commas
- ❑ Asterisk can be used as wildcard character to list all attributes

Updating Table Rows

- ❑ UPDATE - Modify data in a table
 - UPDATE *tablename*
SET *columnname* = *expression* [, *columnname* = *expression*] [WHERE *conditionlist*];
- ❑ If more than one attribute is to be updated in the row, separate corrections with commas
 - UPDATE PRODUCT SET P_INDATE='18-JAN-2004' ,
P_PRICE=16.99 WHERE P_CODE='13-Q2/P2';
 - What would happen if the WHERE condition is omitted?

Restoring Table Contents

❑ ROLLBACK

- Used restore the database to its previous condition
- Only applicable if COMMIT command has not been used to permanently store the changes in the database

❑ Syntax

- ROLLBACK;

❑ COMMIT and ROLLBACK only work with data manipulation commands that are used to add, modify, or delete table rows

Deleting Table Rows

- ❑ DELETE - Deletes a table row
 - DELETE FROM *tablename* [WHERE *conditionlist*];
- ❑ WHERE condition is optional
- ❑ If WHERE condition is not specified, all rows from the specified table will be deleted

Inserting Table Rows with a Select Subquery

☐ INSERT

- Inserts multiple rows from another table (source)
- Uses SELECT subquery
 - ☐ Query that is embedded (or nested) inside another query
 - ☐ Executed first
 - ☐ Values returned should match the attributes and data types of the table in in INSERT statement

☐ Syntax

- INSERT INTO *tablename* SELECT *columnlist* FROM *tablename*

Selecting Rows with Conditional Restrictions

- ❑ Select partial table contents by placing restrictions on rows to be included in output
 - Add conditional restrictions to the SELECT statement, using WHERE clause
- ❑ Syntax
 - SELECT *columnlist*
FROM *tablelist*
[WHERE *conditionlist*] ;

Selected PRODUCT Table Attributes for VENDOR Code 21344

FIGURE 6.5 SELECTED PRODUCT TABLE ATTRIBUTES FOR VENDOR CODE 21344

	P_DESCRIPT	P_INDATE	P_PRICE	V_CODE
►	7.25-in. pwr. saw blade	13-Dec-03	\$14.99	21344
	9.00-in. pwr. saw blade	13-Nov-03	\$17.49	21344
	Rat-tail file, 1/8-in. fine	15-Dec-03	\$4.99	21344

The Microsoft Access QBE and its SQL

FIGURE 6.6 THE MICROSOFT ACCESS QBE AND ITS SQL

The figure illustrates the Microsoft Access Query By Example (QBE) interface and its corresponding SQL statements. The top window shows the QBE grid for a query named 'Microsoft Access - [qryFig6-05 : Select Query]'. The grid displays fields from the 'PRODUCT' table: P_CODE, P_DESCRIPT, P_INDATE, P_ONHAND, P_MIN, P_PRICE, P_DISCOUNT, and V_CODE. The 'Field' row shows the selected fields: P_DESCRIPT, P_INDATE, P_PRICE, and V_CODE. The 'Table' row shows the source table: PRODUCT. The 'Criteria' row shows the filter: V_CODE = 21344. The 'Show' row has checkboxes for each field, with V_CODE checked. The 'Sort' row is empty. The 'Criteria or:' row is empty.

Below the QBE grid, the 'Microsoft Access-generated SQL' section shows the SQL statement generated by Access:

```
SELECT PRODUCT.P_DESCRIPT, PRODUCT.P_INDATE, PRODUCT.P_PRICE, PRODUCT.V_CODE
FROM PRODUCT
WHERE (((PRODUCT.V_CODE)=21344));
```

The bottom section shows the 'User-entered SQL' section, which displays the same SQL statement as the generated SQL:

```
SELECT P_DESCRIPT, P_INDATE, P_PRICE, V_CODE
FROM PRODUCT
WHERE V_CODE=21344;
```

On the right side, the 'View' menu is open, showing options: Design View, SQL View (selected), Datasheet View, PivotTable View, PivotChart View, Totals, Table Names, Properties (F4), Join Properties, and Toolbars.

Comparison Operators

TABLE 6.7 COMPARISON OPERATORS

SYMBOL	MEANING
=	Equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
<> or !=	Not equal to

Selected PRODUCT Table Attributes for VENDOR Codes Other than 21344

FIGURE 6.7 SELECTED PRODUCT TABLE ATTRIBUTES FOR VENDOR CODES OTHER THAN 21344

	P_DESCRIPT	P_INDATE	P_PRICE	V_CODE
►	Power painter, 15 psi., 3-nozzle	03-Nov-03	\$109.99	25595
	Hrd. cloth, 1/4-in., 2x50	15-Jan-04	\$39.95	23119
	Hrd. cloth, 1/2-in., 3x50	15-Jan-04	\$43.99	23119
	B&D jigsaw, 12-in. blade	30-Dec-03	\$109.92	24288
	B&D jigsaw, 8-in. blade	24-Dec-03	\$99.87	24288
	B&D cordless drill, 1/2-in.	20-Jan-04	\$38.95	25595
	Claw hammer	20-Jan-04	\$9.95	21225
	Hicut chain saw, 16 in.	07-Feb-04	\$256.99	24288
	1.25-in. metal screw, 25	01-Mar-04	\$6.99	21225
	2.5-in. wd. screw, 50	24-Feb-04	\$8.45	21231
	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-04	\$119.95	25595

Selected PRODUCT Table Attributes with a P_PRICE Restriction

FIGURE 6.8 SELECTED PRODUCT TABLE ATTRIBUTES WITH A P_PRICE RESTRICTION

P_PRICE ≤ 10

	P_DESCRIPTION	P_ONHAND	P_MIN	P_PRICE
►	Claw hammer	23	10	\$9.95
	Rat-tail file, 1/8-in. fine	43	20	\$4.99
	PVC pipe, 3.5-in., 8-ft	188	75	\$5.87
	1.25-in. metal screw, 25	172	75	\$6.99
	2.5-in. wd. screw, 50	237	100	\$8.45

Selected PRODUCT Table Attributes: The ASCII Code Effect

FIGURE 6.9 SELECTED PRODUCT TABLE ATTRIBUTES: THE ASCII CODE EFFECT

P_CODE <
'15558-QW1'

	P_CODE	P_DESCRIPT	P_ONHAND	P_MIN	P_PRICE
►	11QER/31	Power painter, 15 psi., 3-nozzle	8	5	\$109.99
	13-Q2/P2	7.25-in. pwr. saw blade	32	15	\$14.99
	14-Q1/L3	9.00-in. pwr. saw blade	18	12	\$17.49
	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15	8	\$39.95

Selected PRODUCT Table Attributes: Date Restriction

FIGURE 6.10 SELECTED PRODUCT TABLE ATTRIBUTES: DATE RESTRICTION

P_INDATE >= '20-
Jan-2004'

Access uses # for
date delimiter

	P_DESCRIPT	P_ONHAND	P_MIN	P_PRICE	P_INDATE
▶	B&D cordless drill, 1/2-in.	12	5	\$38.95	20-Jan-04
	Claw hammer	23	10	\$9.95	20-Jan-04
	Hicut chain saw, 16 in.	11	5	\$256.99	07-Feb-04
	PVC pipe, 3.5-in., 8-ft	188	75	\$5.87	20-Feb-04
	1.25-in. metal screw, 25	172	75	\$6.99	01-Mar-04
	2.5-in. wd. screw, 50	237	100	\$8.45	24-Feb-04

SELECT Statement with a Computed Column

FIGURE 6.11 SELECT STATEMENT WITH A COMPUTED COLUMN

```
SELECT  
P_ONHAND*P_PRICE  
FROM PRODUCT
```

	P_DESCRIPTOR	P_ONHAND	P_PRICE	Expr1
►	Power painter, 15 psi., 3-nozzle	8	\$109.99	\$879.92
	7.25-in. pwr. saw blade	32	\$14.99	\$479.68
	9.00-in. pwr. saw blade	18	\$17.49	\$314.82
	Hrd. cloth, 1/4-in., 2x50	15	\$39.95	\$599.25
	Hrd. cloth, 1/2-in., 3x50	23	\$43.99	\$1,011.77
	B&D jigsaw, 12-in. blade	8	\$109.92	\$879.36
	B&D jigsaw, 8-in. blade	6	\$99.87	\$599.22
	B&D cordless drill, 1/2-in.	12	\$38.95	\$467.40
	Claw hammer	23	\$9.95	\$228.85
	Sledge hammer, 12 lb.	8	\$14.40	\$115.20
	Rat-tail file, 1/8-in. fine	43	\$4.99	\$214.57
	Hicut chain saw, 16 in.	11	\$256.99	\$2,826.89
	PVC pipe, 3.5-in., 8-ft	188	\$5.87	\$1,103.56
	1.25-in. metal screw, 25	172	\$6.99	\$1,202.28
	2.5-in. wd. screw, 50	237	\$8.45	\$2,002.65
	Steel matting, 4'x8'x1/8", .5" mesh	18	\$119.95	\$2,159.10

SELECT Statement with a Computed Column and an Alias

FIGURE 6.12 SELECT STATEMENT WITH A COMPUTED COLUMN AND AN ALIAS

```
SELECT  
P_ONHAND*P_PRICE  
AS TOTVALUE FROM  
PRODUCT
```

	P_DESCRIPTION	P_ONHAND	P_PRICE	TOTVALUE
▶	Power painter, 15 psi., 3-nozzle	8	\$109.99	\$879.92
	7.25-in. pwr. saw blade	32	\$14.99	\$479.68
	9.00-in. pwr. saw blade	18	\$17.49	\$314.82
	Hrd. cloth, 1/4-in., 2x50	15	\$39.95	\$599.25
	Hrd. cloth, 1/2-in., 3x50	23	\$43.99	\$1,011.77
	B&D jigsaw, 12-in. blade	8	\$109.92	\$879.36
	B&D jigsaw, 8-in. blade	6	\$99.87	\$599.22
	B&D cordless drill, 1/2-in.	12	\$38.95	\$467.40
	Claw hammer	23	\$9.95	\$228.85
	Sledge hammer, 12 lb.	8	\$14.40	\$115.20
	Rat-tail file, 1/8-in. fine	43	\$4.99	\$214.57
	Hicut chain saw, 16 in.	11	\$256.99	\$2,826.89
	PVC pipe, 3.5-in., 8-ft	188	\$5.87	\$1,103.56
	1.25-in. metal screw, 25	172	\$6.99	\$1,202.28
	2.5-in. wd. screw, 50	237	\$8.45	\$2,002.65
	Steel matting, 4'x8'x1/8", .5" mesh	18	\$119.95	\$2,159.10

SELECT statement: date computations

- ❑ `SELECT P_CODE, P_INDATE, DATE()-90 AS CUTDATE FROM PRODUCT WHERE P_INDATE <=DATE()-90;`
- ❑ `SELECT P_CODE, P_INDATE, P_INDATE+90 AS EXPDATE FROM PRODUCT;`

Arithmetic Operators: The Rule of Precedence

- ❑ Perform operations within parentheses
- ❑ Perform power operations
- ❑ Perform multiplications and divisions
- ❑ Perform additions and subtractions

Selected PRODUCT Table Attributes: The Logical OR

FIGURE 6.13 SELECTED PRODUCT TABLE ATTRIBUTES: THE LOGICAL OR

V_CODE=21344
OR
V_CODE=24288

	P_DESCRIPT	P_INDATE	P_PRICE	V_CODE
▶	7.25-in. pwr. saw blade	13-Dec-03	\$14.99	21344
	9.00-in. pwr. saw blade	13-Nov-03	\$17.49	21344
	B&D jigsaw, 12-in. blade	30-Dec-03	\$109.92	24288
	B&D jigsaw, 8-in. blade	24-Dec-03	\$99.87	24288
	Rat-tail file, 1/8-in. fine	15-Dec-03	\$4.99	21344
	Hicut chain saw, 16 in.	07-Feb-04	\$256.99	24288

Selected PRODUCT Table Attributes: The Logical AND

FIGURE 6.14 SELECTED PRODUCT TABLE ATTRIBUTES: THE LOGICAL AND

**P_PRICE < 50
AND
P_INDATE >
'15-Jan-2004'**

	P_DESCRIPT	P_INDATE	P_PRICE	V_CODE
▶	B&D cordless drill, 1/2-in.	20-Jan-04	\$38.95	25595
	Claw hammer	20-Jan-04	\$9.95	21225
	PVC pipe, 3.5-in., 8-ft	20-Feb-04	\$5.87	
	1.25-in. metal screw, 25	01-Mar-04	\$6.99	21225
	2.5-in. wdl. screw, 50	24-Feb-04	\$8.45	21231

Selected PRODUCT Table Attributes: The Logical AND and OR

FIGURE 6.15 SELECTED PRODUCT TABLE ATTRIBUTES: THE LOGICAL AND AND OR

**(P_PRICE <50
AND
P_INDATE >
'15-Jan-2004')
OR
V_CODE=24288**

	P_DESCRIPTOR	P_INDATE	P_PRICE	V_CODE
▶	B&D jigsaw, 12-in. blade	30-Dec-03	\$109.92	24288
	B&D jigsaw, 8-in. blade	24-Dec-03	\$99.87	24288
	B&D cordless drill, 1/2-in.	20-Jan-04	\$38.95	25595
	Claw hammer	20-Jan-04	\$9.95	21225
	Hicut chain saw, 16 in.	07-Feb-04	\$256.99	24288
	PVC pipe, 3.5-in., 8-ft	20-Feb-04	\$5.87	
	1.25-in. metal screw, 25	01-Mar-04	\$6.99	21225
	2.5-in. wd. screw, 50	24-Feb-04	\$8.45	21231

Special Operators

❑ **BETWEEN**

- Used to check whether attribute value is within a range `P_PRICE BETWEEN 50.00 AND 100.00`

❑ **IS NULL**

- Used to check whether attribute value is null

❑ **LIKE**

- Used to check whether attribute value matches a given string pattern `V_CONTACT [NOT] LIKE 'Smith*'`
- MS Access uses `*` and `?` instead of `%` and `_`

Special Operators

IN

- Used to check whether attribute value matches any value within a value list
 -  **V_CODE IN (21344,24288) if numeric**
 -  **SELECT V_CODE, V_NAME FROM VENDOR
WHERE V_CODE IN
(SELECT V_CODE FROM PRODUCT)**

EXISTS

- Used to check if a subquery returns any rows
 -  **SELECT * FROM VENDOR
WHERE EXISTS
(SELECT * FROM PRODUCT
WHERE P_ONHAND <=P_MIN)**

Advanced Data Definition Commands

- ❑ All changes in the table structure are made by using the ALTER command
 - Followed by a keyword that produces specific change
 - Three options are available
 - ❑ ADD
 - ❑ MODIFY
 - ❑ DROP

Changing a Column's Data Type

- ❑ ALTER can be used to change the data type from integer to character
 - `ALTER TABLE PRODUCT`
`MODIFY(V_CODE CHAR(5));`
 - If V_CODE in PRODUCT references the same field in VENDOR, there will be a referential integrity violation
- ❑ Some RDBMSs (such as Oracle) do not permit changes to data types unless the column to be changed is empty

Changing a Column's Data Characteristics

- ❑ Use ALTER to change data characteristics – increase the width of P_PRICE
 - `ALTER TABLE PRODUCT
MODIFY(P_PRICE DECIMAL(9,2));`
- ❑ If the column to be changed already contains data, changes in the column's characteristics are permitted if those changes do not alter the data type
- ❑ Some databases will allow you to increase but not decrease the width of a column

Adding or Dropping a Column

- ❑ Use ALTER to add a column
 - ALTER TABLE PRODUCT
ADD(P_SALECODE CHAR(1));
 - Do not include the NOT NULL clause for new column because when it is added to an existing row it will default to NULL. It can be added to the table structure after all the data has been entered

- ❑ Use ALTER to drop a column
 - ALTER TABLE PRODUCT
DROP COLUMN V_ORDER;
 - Some RDBMSs impose restrictions on the deletion of an attribute e.g., if involved in a FK relationship

The Effect of Data Entry into the New P_SALECODE Column

UPDATE PRODUCT

SET P_SALECODE = '2'

WHERE P_CODE = '1546-QQ2'

FIGURE 6.16 THE EFFECT OF DATA ENTRY INTO THE NEW P_SALECODE COLUMN

	P_CODE	P_DESCRIPT	P_INDATE	P_ONHAND	P_MIN	P_PRICE	P_DISCOUNT	V_CODE	P_SALECODE
▶	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-04	15	8	\$39.95	0.00	23119	2

Update of the P_SALECODE Column in Multiple Data Rows

FIGURE 6.17 UPDATE OF THE P_SALECODE COLUMN IN MULTIPLE DATA ROWS

```
UPDATE PRODUCT  
SET P_SALECODE = '1'  
WHERE P_CODE IN  
( '2232/QWE',  
  '2232/QTY');
```

	P_CODE	P_DESCRIPT	P_INDATE	P_PRICE	P_SALECODE
►	11QER/31	Power painter, 15 psi., 3-	03-Nov-03	\$109.99	
	13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-03	\$14.99	
	14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-03	\$17.49	
	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-04	\$39.95	2
	1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-04	\$43.99	
	2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-03	\$109.92	1
	2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-03	\$99.87	1
	2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-04	\$38.95	
	23109-HB	Claw hammer	20-Jan-04	\$9.95	
	23114-AA	Sledge hammer, 12 lb.	02-Jan-04	\$14.40	
	54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-03	\$4.99	
	89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-04	\$256.99	
	PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-04	\$5.87	
	SM-18277	1.25-in. metal screw, 25	01-Mar-04	\$6.99	
	SW-23116	2.5-in. wdl. screw, 50	24-Feb-04	\$8.45	
	WR3/TT3	Steel matting, 4'x8'x1/6",	17-Jan-04	\$119.95	

The Effect of Multiple Data Updates in the PRODUCT Table

FIGURE 6.18 THE EFFECT OF MULTIPLE DATA UPDATES IN THE PRODUCT TABLE (MS ACCESS)

```
UPDATE PRODUCT  
SET P_SALECODE = '2'  
WHERE P_INDATE  
<'25-Dec-2003';
```

```
UPDATE PRODUCT  
SET P_SALECODE = '1'  
WHERE P_INDATE  
>='16-Jan-2004'  
AND  
P_INDATE  
<='10-Feb-2004';
```

	P_CODE	P_DESCRIPT	P_INDATE	P_PRICE	P_SALECODE
▶	11QER/31	Power painter, 15 psi., 3-nozzle	33-Nov-2003	\$109.99	2
	13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-2003	\$14.99	2
	14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-2003	\$17.49	2
	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-2004	\$39.95	
	1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-2004	\$43.99	
	2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-2003	\$109.92	
	2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-2003	\$99.87	2
	2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-2004	\$38.95	1
	23109-HB	Claw hammer	20-Jan-2004	\$9.95	1
	23114-AA	Sledge hammer, 12 lb.	02-Jan-2004	\$14.40	
	54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-2003	\$4.99	2
	89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-2004	\$256.99	1
	PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-2004	\$5.87	
	SM-18277	1.25-in. metal screw, 25	01-Mar-2004	\$6.99	
	SW-23116	2.5-in. wd. screw, 50	24-Feb-2004	\$8.45	
	WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-2004	\$119.95	1

Updates with Arithmetic Operators

UPDATE PRODUCT

**SET P_ONHAND = P_ONHAND+2
WHERE P_CODE='2232/QWE';**

UPDATE PRODUCT

**SET P_PRICE = P_PRICE*1.10
WHERE P_PRICE < 50.00;**

Copying Parts of Tables

- ❑ SQL permits copying contents of selected table columns so that the data need not be reentered manually into newly created table(s)
- ❑ First create the PART table structure
 - Row names need not be the same nor the number of columns as the original table but those columns being copied must have the same attributes
- ❑ Next add rows to new PART table using PRODUCT table rows

PART Attributes Copied from the PRODUCT Table

FIGURE 6.19 PART ATTRIBUTES COPIED FROM THE PRODUCT TABLE

```
INSERT INTO PART  
(PART_CODE,  
PART_DESCRIP,  
PART_PRICE)  
SELECT P_CODE,  
P_DESCRIP,  
P_PRICE  
FROM PRODUCT;
```

	PART_CODE	PART_DESCRIP	PART_PRICE
▶	11QER/31	Power painter, 15 psi., 3-nozzle	\$109.99
	13-Q2/P2	7.25-in. pwr. saw blade	\$14.99
	14-Q1/L3	9.00-in. pwr. saw blade	\$17.49
	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	\$39.95
	1558-QW1	Hrd. cloth, 1/2-in., 3x50	\$43.99
	2232/QTY	B&D jigsaw, 12-in. blade	\$109.92
	2232/QWE	B&D jigsaw, 8-in. blade	\$99.87
	2238/QPD	B&D cordless drill, 1/2-in.	\$38.95
	23109-HB	Claw hammer	\$9.95
	23114-AA	Sledge hammer, 12 lb.	\$14.40
	54778-2T	Rat-tail file, 1/8-in. fine	\$4.99
	89-WRE-Q	Hicut chain saw, 16 in.	\$256.99
	PVC23DRT	PVC pipe, 3.5-in., 8-ft	\$5.87
	SM-18277	1.25-in. metal screw, 25	\$6.99
	SW-23116	2.5-in. wd. screw, 50	\$8.45
	WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	\$119.95

Creating the PART table and Copying from the PRODUCT in One Step

```
CREATE TABLE PART AS  
SELECT
```

```
    P_CODE AS PART_CODE,  
    P_DESCRIPT AS PART_DESCRIPT,  
    P_PRICE AS PART_PRICE
```

```
FROM PRODUCT;
```

- ☐ Column names need not be identical
- ☐ New table need not have the same number of columns as the original
- ☐ No entity integrity (PK) or referential integrity (FK) rules are automatically applied to the new table

Additional SQL Operations

- ❑ You can add primary (single and composite) and foreign keys using the ALTER command

ALTER TABLE LINE

```
ADD PRIMARY KEY(INV_NUMBER,LINE_NUMBER),  
ADD FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE,  
ADD FOREIGN KEY (PROD_CODE) REFERENCES PRODUCT;
```

- ❑ Deleting a table from the database

```
DROP TABLE PART;
```

Additional SQL Operations

- ❑ Ordering a listing

```
SELECT P_CODE, P_DESCRIPT, P_INDATE, P_PRICE  
FROM PRODUCT  
ORDER BY P_PRICE;
```

ORDER BY P_PRICE DESC puts the listing in descending order

- ❑ Cascading order sequence – multilevel ordering
 - You can also include a WHERE clause before the ORDER clause

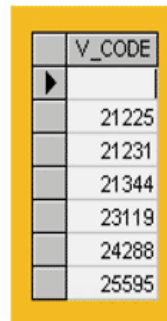
```
SELECT EMP_LNAME, EMP_FNAME, EMP_INITIAL,  
EMP_AREACODE, EMP_PHONE FROM EMPLOYEE  
WHERE EMP_AREACODE IN ('718', '212')  
ORDER BY  
EMP_LNAME, EMP_FNAME, EMP_INITIAL;
```

Additional SQL Operations

- ❑ Listing unique values – find out how many different vendors are in the PRODUCT table

SELECT DISTINCT V_CODE FROM PRODUCT;

FIGURE 6.24 A LISTING OF DISTINCT (DIFFERENT) V_CODE VALUES IN THE PRODUCT TABLE

A screenshot of a database query result. It shows a table with a single column labeled 'V_CODE'. The table contains seven rows of data, each with a unique numerical value. The values are 21225, 21231, 21344, 23119, 24288, and 25595. The table is displayed within a yellow-bordered window.

V_CODE
21225
21231
21344
23119
24288
25595

Advanced Select Queries

- ❑ SQL provides useful functions
 - Count
 - Sum
 - Find minimum and maximum values
 - Calculate averages
- ❑ SQL allows the user to limit queries to only those entries having no duplicates or entries whose duplicates may be grouped

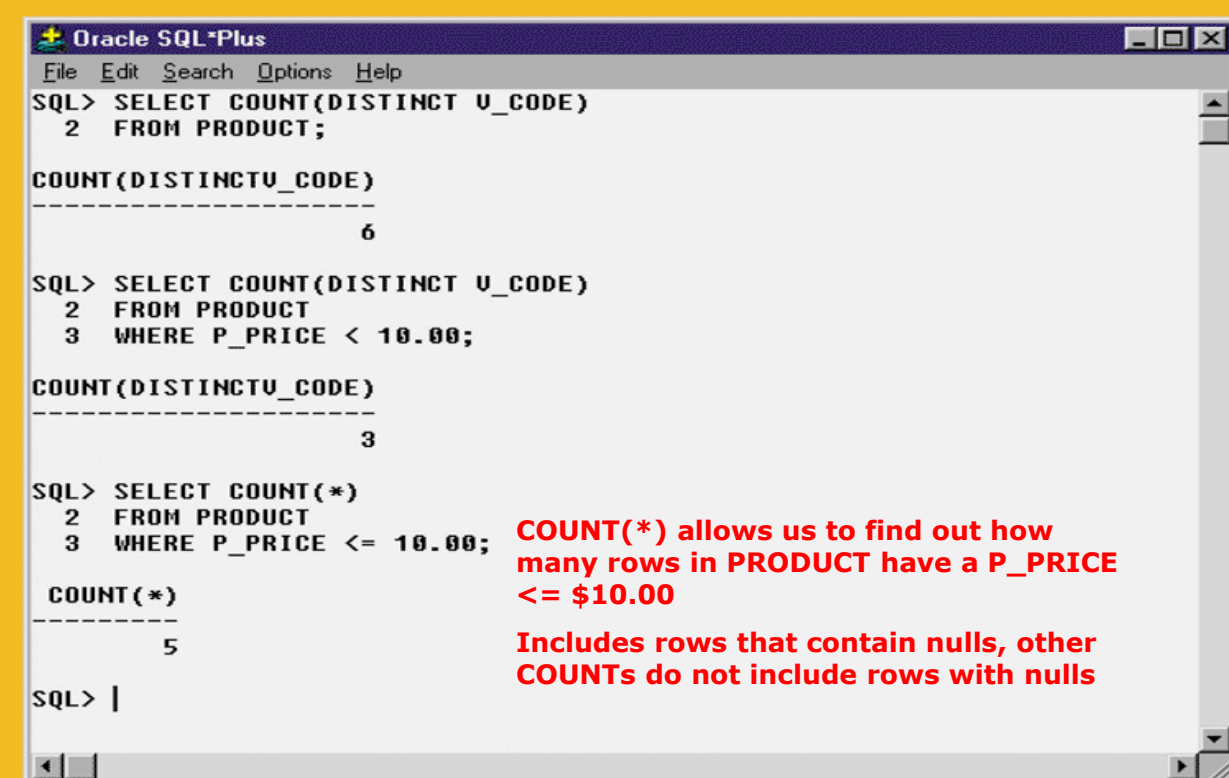
Some Basic SQL Aggregate Functions

TABLE 6.10 SOME BASIC SQL AGGREGATE FUNCTIONS

FUNCTION	OUTPUT
COUNT	The number of rows containing “non null” values
MIN	The minimum attribute value encountered in a given column
MAX	The maximum attribute value encountered in a given column
SUM	The sum of all values for a given column
AVG	The arithmetic mean (average) for the specified column

COUNT Function Output Examples

FIGURE 6.25 COUNT FUNCTION OUTPUT EXAMPLES



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT COUNT(DISTINCT V_CODE)
2 FROM PRODUCT;

COUNT(DISTINCT V_CODE)
-----
6

SQL> SELECT COUNT(DISTINCT V_CODE)
2 FROM PRODUCT
3 WHERE P_PRICE < 10.00;

COUNT(DISTINCT V_CODE)
-----
3

SQL> SELECT COUNT(*)
2 FROM PRODUCT
3 WHERE P_PRICE <= 10.00;

COUNT(*)
-----
5

SQL> |
```

COUNT(*) allows us to find out how many rows in PRODUCT have a P_PRICE <= \$10.00

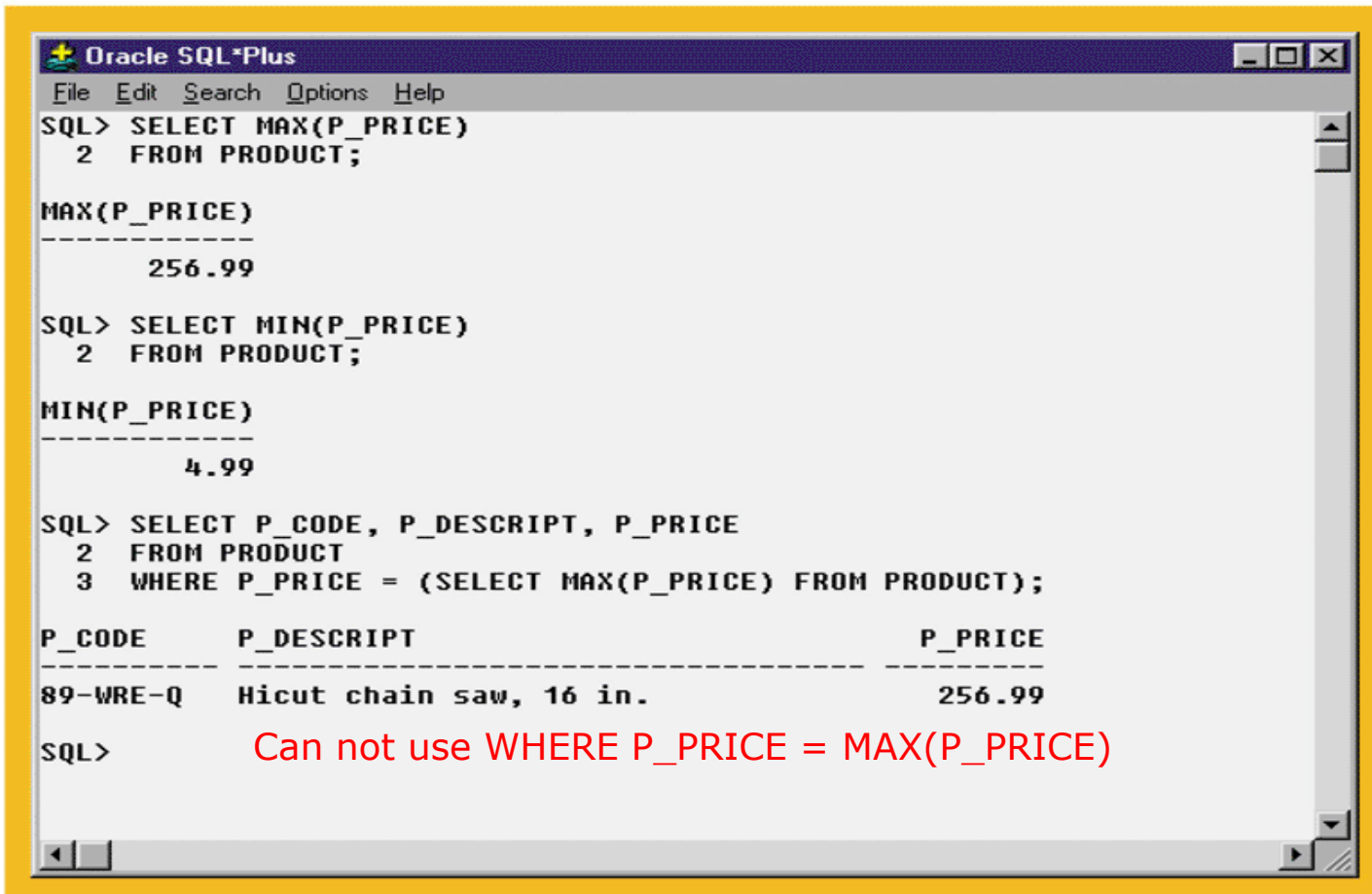
Includes rows that contain nulls, other COUNTs do not include rows with nulls

MS Access does not allow the use of COUNT with DISTINCT

Use `SELECT COUNT(*) FROM SELECT DISTINCT V_CODE WHERE V_CODE IS NOT NULL`

MAX and MIN Function Output

FIGURE 6.26 MAX AND MIN FUNCTION OUTPUT EXAMPLES



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT MAX(P_PRICE)
2 FROM PRODUCT;

MAX(P_PRICE)
-----
      256.99

SQL> SELECT MIN(P_PRICE)
2 FROM PRODUCT;

MIN(P_PRICE)
-----
       4.99

SQL> SELECT P_CODE, P_DESCRIPT, P_PRICE
2 FROM PRODUCT
3 WHERE P_PRICE = (SELECT MAX(P_PRICE) FROM PRODUCT);

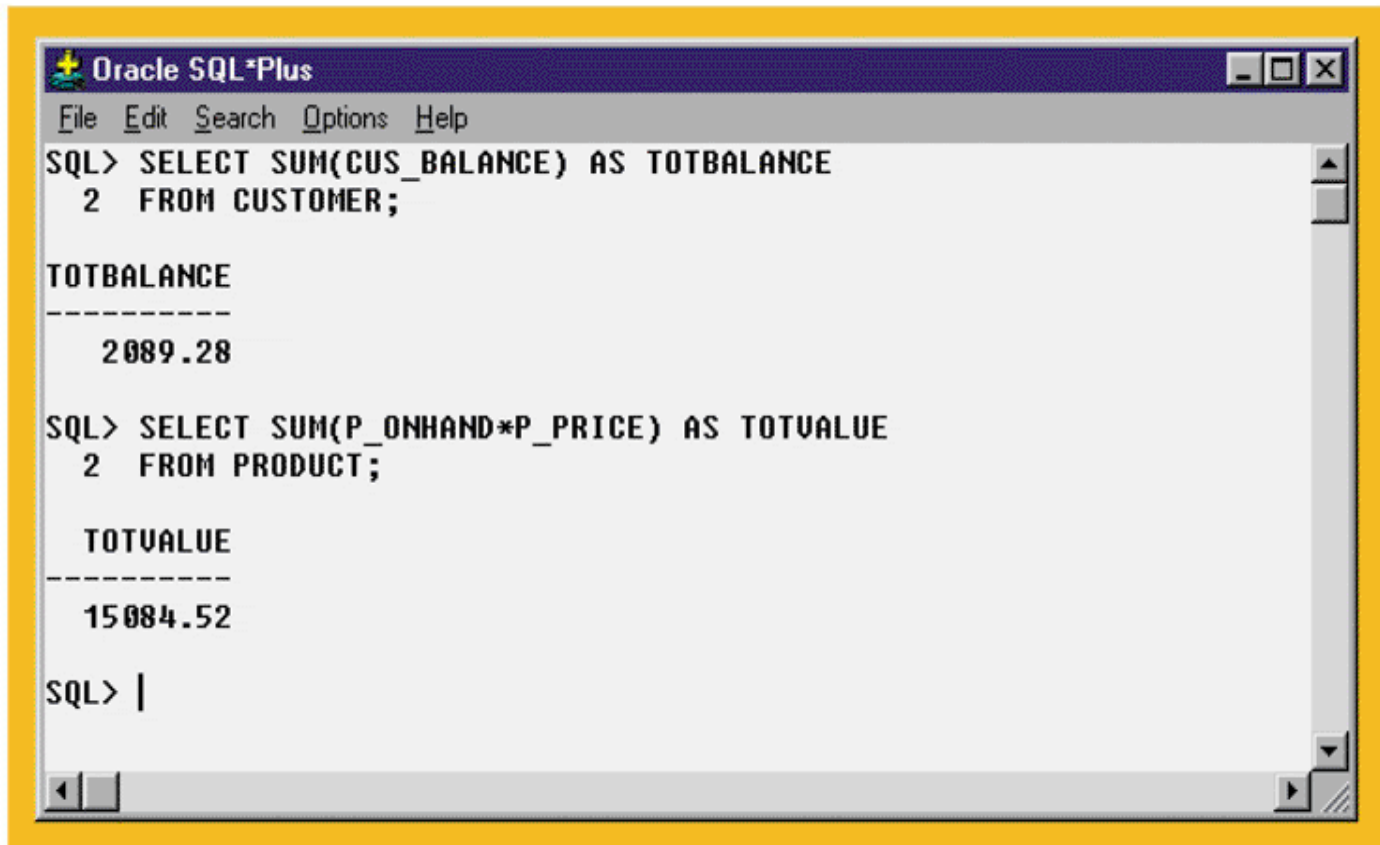
P_CODE      P_DESCRIPT      P_PRICE
-----
89-WRE-Q    Hicut chain saw, 16 in.      256.99

SQL>
```

Can not use WHERE P_PRICE = MAX(P_PRICE)

The Total Value of All Items in the PRODUCT Table

FIGURE 6.27 THE TOTAL VALUE OF ALL ITEMS IN THE PRODUCT TABLE

A screenshot of the Oracle SQL*Plus command-line interface. The window has a title bar 'Oracle SQL*Plus' and a menu bar with 'File', 'Edit', 'Search', 'Options', and 'Help'. The main text area shows two SQL queries and their results. The first query is 'SELECT SUM(CUS_BALANCE) AS TOTBALANCE' followed by a blank line and 'FROM CUSTOMER;'. The result is a single row with the value '2089.28' under the column header 'TOTBALANCE'. The second query is 'SELECT SUM(P_ONHAND*P_PRICE) AS TOTVALUE' followed by a blank line and 'FROM PRODUCT;'. The result is a single row with the value '15084.52' under the column header 'TOTVALUE'. The prompt 'SQL>' is followed by a vertical bar cursor.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT SUM(CUS_BALANCE) AS TOTBALANCE
      2  FROM CUSTOMER;

TOTBALANCE
-----
      2089.28

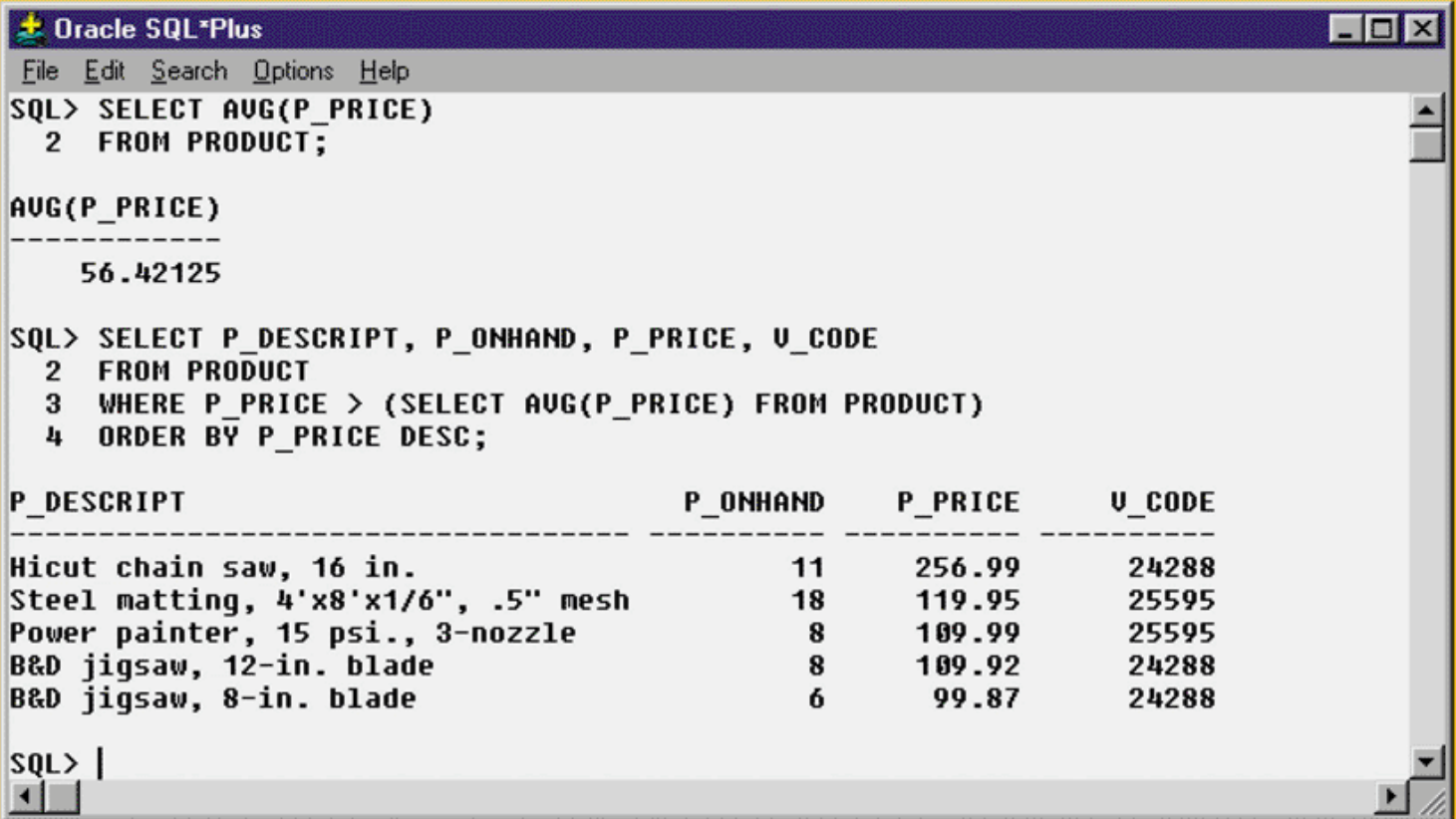
SQL> SELECT SUM(P_ONHAND*P_PRICE) AS TOTVALUE
      2  FROM PRODUCT;

TOTVALUE
-----
     15084.52

SQL> |
```

AVG Function Output Examples

FIGURE 6.28 AVG FUNCTION OUTPUT EXAMPLES



The screenshot shows the Oracle SQL*Plus interface with two SQL queries and their results. The first query calculates the average price of products, and the second query lists products with prices above the average, ordered by price in descending order.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT AVG(P_PRICE)
  2  FROM PRODUCT;

AVG(P_PRICE)
-----
56.42125

SQL> SELECT P_DESCRIPTOR, P_ONHAND, P_PRICE, U_CODE
  2  FROM PRODUCT
  3  WHERE P_PRICE > (SELECT AVG(P_PRICE) FROM PRODUCT)
  4  ORDER BY P_PRICE DESC;

P_DESCRIPTOR                                P_ONHAND    P_PRICE    U_CODE
-----
Hicut chain saw, 16 in.                      11      256.99     24288
Steel matting, 4'x8'x1/6", .5" mesh          18      119.95     25595
Power painter, 15 psi., 3-nozzle              8      109.99     25595
B&D jigsaw, 12-in. blade                      8      109.92     24288
B&D jigsaw, 8-in. blade                       6       99.87     24288

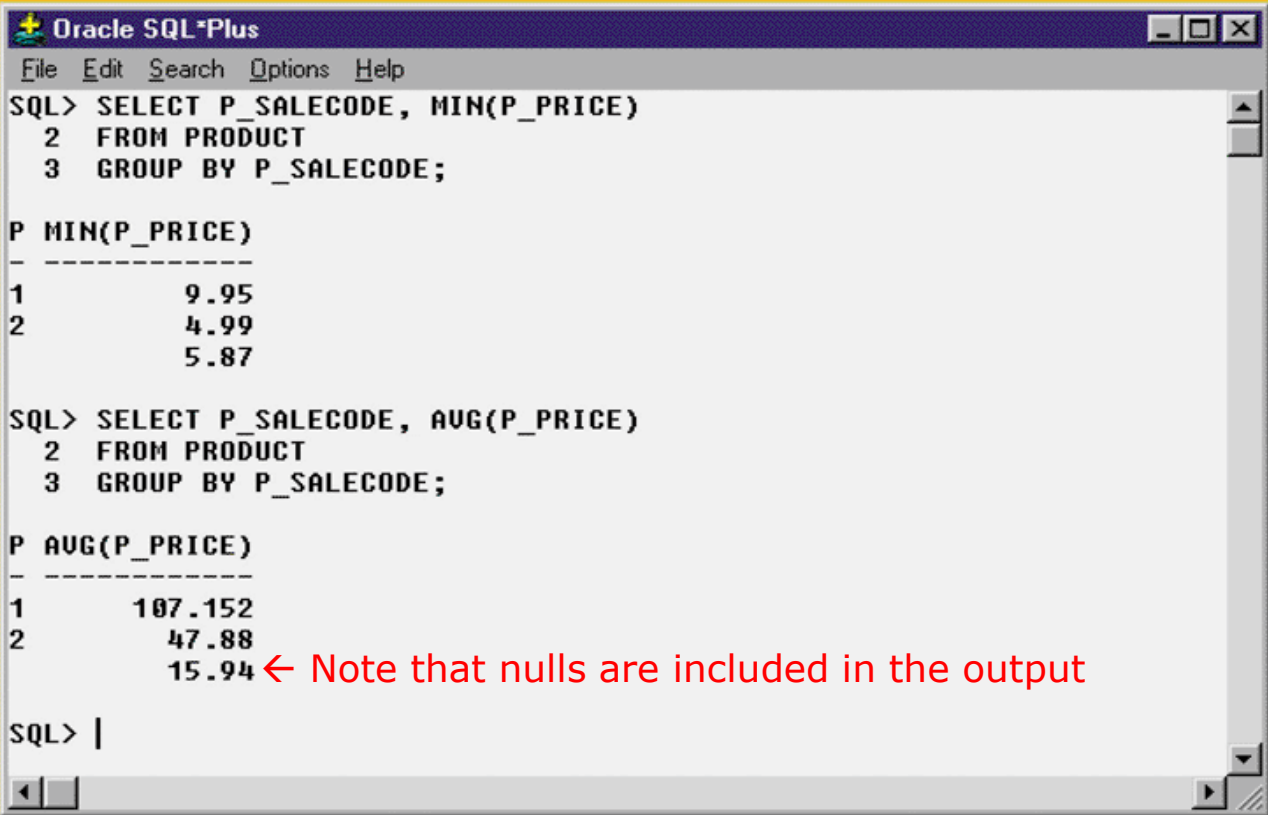
SQL> |
```

GROUP BY Clause

- ❑ The SELECT's columnlist must include a combination of column names and aggregate functions
- ❑ The GROUP BY clause's columnlist must include all non-aggregate function columns specified in the SELECT's columnlist.
 - If required, you could also group by an aggregate function columns that appear in the SELECT's columnlist
- ❑ The GROUP BY clause columnlist can include any columns from the tables in the FROM clause of the SELECT statement, even if they do not appear in the SELECT columnlist

GROUP BY Clause Output Examples

FIGURE 6.29 GROUP BY CLAUSE OUTPUT EXAMPLES



The screenshot shows the Oracle SQL*Plus interface with a menu bar (File, Edit, Search, Options, Help) and a command window. The first query is: SQL> SELECT P_SALECODE, MIN(P_PRICE) FROM PRODUCT GROUP BY P_SALECODE;. The output shows three rows for P_SALECODE 1, 2, and 3, with their respective minimum prices. The second query is: SQL> SELECT P_SALECODE, AVG(P_PRICE) FROM PRODUCT GROUP BY P_SALECODE;. The output shows three rows for P_SALECODE 1, 2, and 3, with their respective average prices. A red arrow points to the third row of the second query's output, with the text 'Note that nulls are included in the output'.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT P_SALECODE, MIN(P_PRICE)
2 FROM PRODUCT
3 GROUP BY P_SALECODE;

P MIN(P_PRICE)
-----
1          9.95
2          4.99
3          5.87

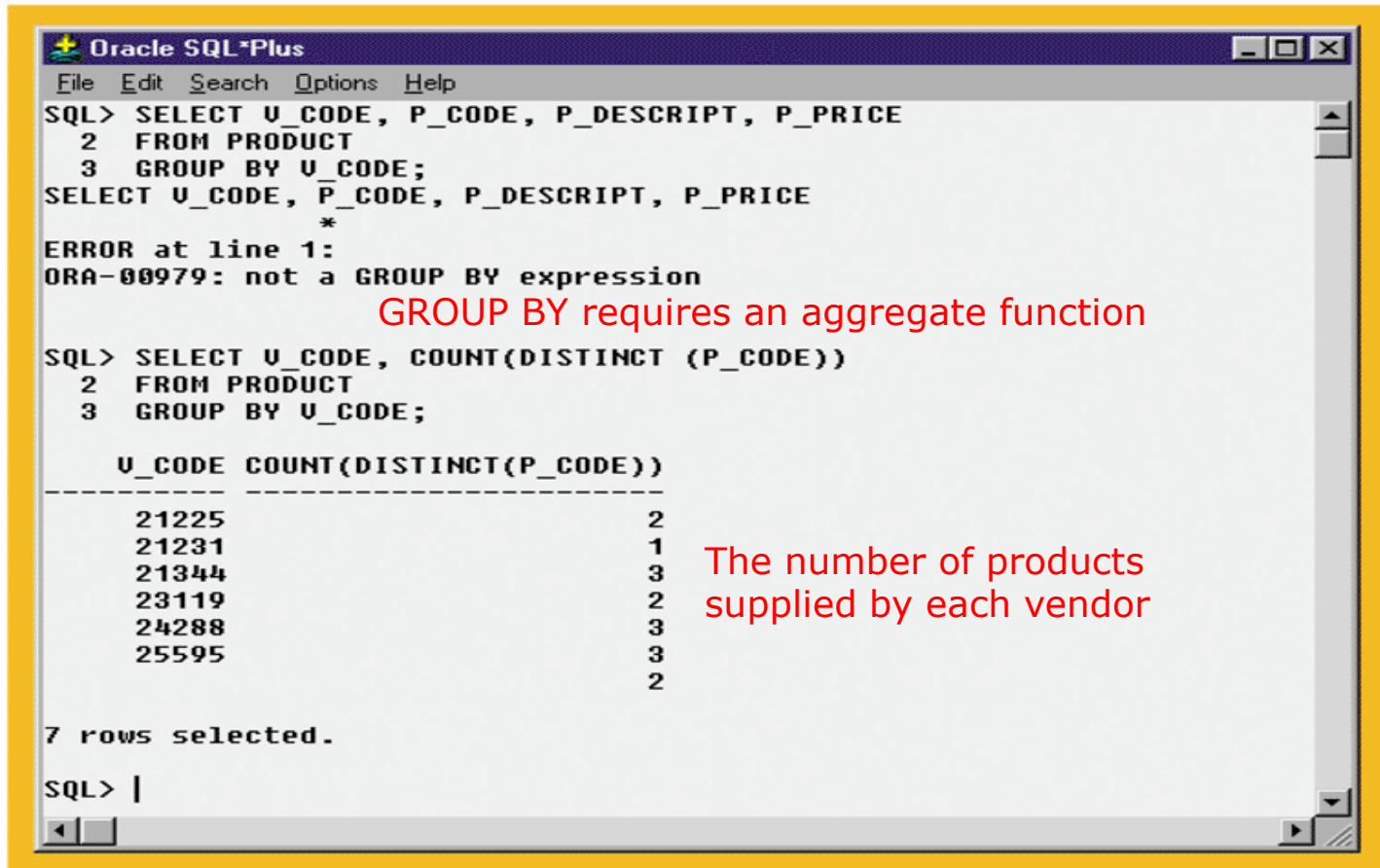
SQL> SELECT P_SALECODE, AVG(P_PRICE)
2 FROM PRODUCT
3 GROUP BY P_SALECODE;

P AVG(P_PRICE)
-----
1        107.152
2         47.88
3         15.94 ← Note that nulls are included in the output

SQL> |
```

Incorrect and Correct Use of the GROUP BY Clause

FIGURE 6.30 INCORRECT AND CORRECT USE OF THE GROUP BY CLAUSE



The screenshot shows the Oracle SQL*Plus interface. The first query is incorrect, attempting to group by a non-aggregate column. The second query is correct, using an aggregate function with the GROUP BY clause. The result of the second query is displayed as a table.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT U_CODE, P_CODE, P_DESCRIPT, P_PRICE
2 FROM PRODUCT
3 GROUP BY U_CODE;
SELECT U_CODE, P_CODE, P_DESCRIPT, P_PRICE
      *
ERROR at line 1:
ORA-00979: not a GROUP BY expression
```

GROUP BY requires an aggregate function

```
SQL> SELECT U_CODE, COUNT(DISTINCT (P_CODE))
2 FROM PRODUCT
3 GROUP BY U_CODE;
```

U_CODE	COUNT(DISTINCT(P_CODE))
21225	2
21231	1
21344	3
23119	2
24288	3
25595	3
	2

7 rows selected.

The number of products supplied by each vendor

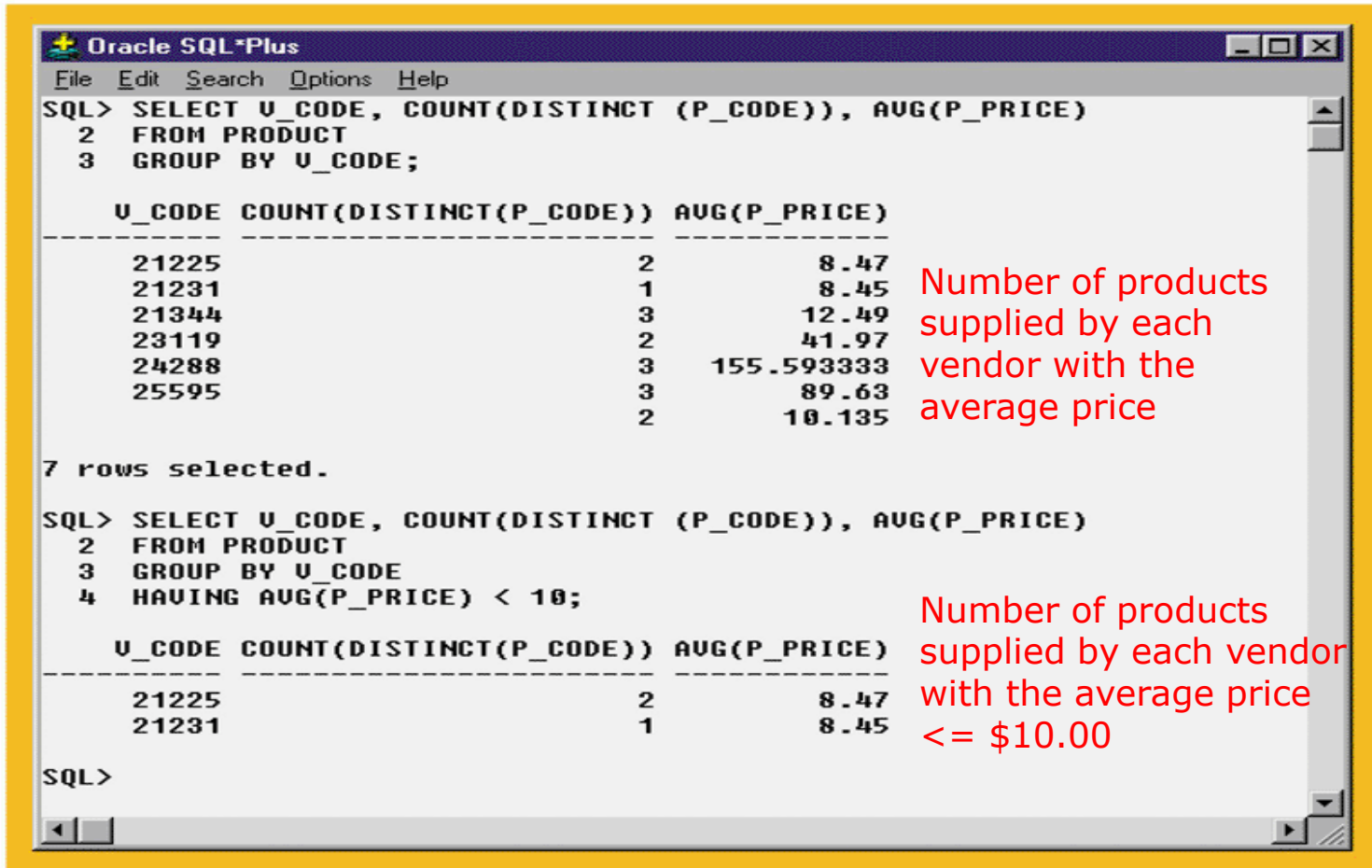
```
SQL> |
```

An Application of the HAVING Clause

- ❑ HAVING operates like the WHERE clause in the SELECT statement except...
 - WHERE applies to columns and expressions for individual rows
 - HAVING is applied to the output of a GROUP operation

An Application of the HAVING Clause

FIGURE 6.31 AN APPLICATION OF THE HAVING CLAUSE



Oracle SQL*Plus

```
File Edit Search Options Help
SQL> SELECT U_CODE, COUNT(DISTINCT (P_CODE)), AVG(P_PRICE)
2 FROM PRODUCT
3 GROUP BY U_CODE;
```

U_CODE	COUNT(DISTINCT(P_CODE))	AVG(P_PRICE)
21225	2	8.47
21231	1	8.45
21344	3	12.49
23119	2	41.97
24288	3	155.593333
25595	3	89.63
	2	10.135

7 rows selected.

```
SQL> SELECT U_CODE, COUNT(DISTINCT (P_CODE)), AVG(P_PRICE)
2 FROM PRODUCT
3 GROUP BY U_CODE
4 HAVING AVG(P_PRICE) <= 10;
```

U_CODE	COUNT(DISTINCT(P_CODE))	AVG(P_PRICE)
21225	2	8.47
21231	1	8.45

SQL>

Number of products supplied by each vendor with the average price

Number of products supplied by each vendor with the average price <= \$10.00

Virtual Tables: Creating a View

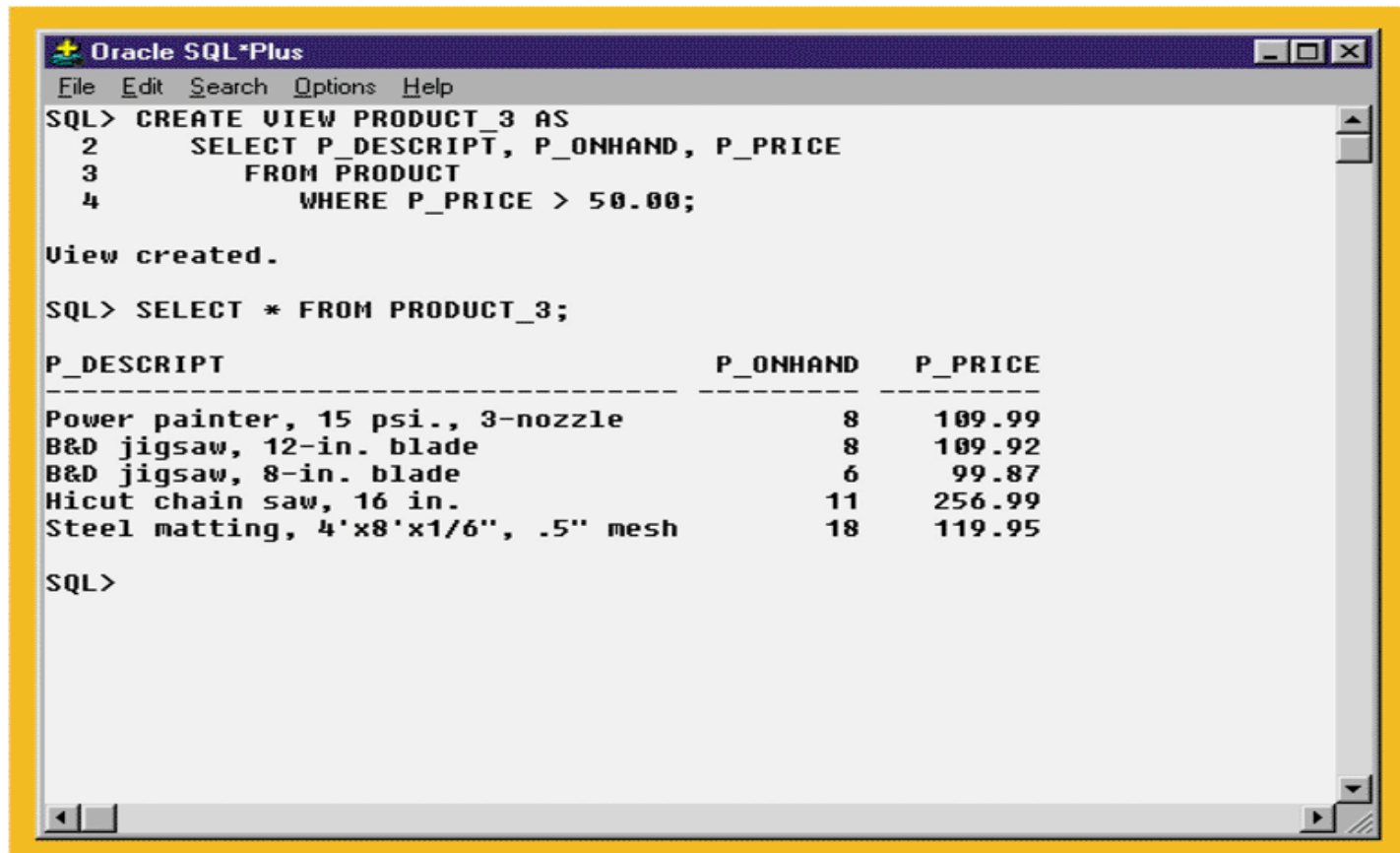
- ❑ View is a *virtual table* based on a SELECT query
 - Can contain columns, computed columns, aliases, and aggregate functions from one or more tables
- ❑ Base tables are tables on which the view is based
- ❑ Create a view by using the CREATE VIEW command
 - Not supported in MS Access

Virtual Tables: Creating a View

- ❑ You can use the name of a view anywhere a table is expected in a SQL statement
- ❑ Views are dynamically updated – it is recreated each time it is invoked
- ❑ Views can be restrict users to see only specified columns and rows in a table
- ❑ They can be used as the basis of a report
 - `CREATE VIEW XYZ AS SELECT ... FROM ... GROUP BY ...;`

Creating a Virtual Table with the CREATE VIEW Command

FIGURE 6.32 CREATING A VIRTUAL TABLE WITH THE CREATE VIEW COMMAND



The screenshot shows the Oracle SQL*Plus interface. The command window contains the following text:

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE VIEW PRODUCT_3 AS
2     SELECT P_DESCRIPT, P_ONHAND, P_PRICE
3     FROM PRODUCT
4     WHERE P_PRICE > 50.00;

View created.

SQL> SELECT * FROM PRODUCT_3;
```

P_DESCRIPT	P_ONHAND	P_PRICE
Power painter, 15 psi., 3-nozzle	8	109.99
B&D jigsaw, 12-in. blade	8	109.92
B&D jigsaw, 8-in. blade	6	99.87
Hicut chain saw, 16 in.	11	256.99
Steel matting, 4'x8'x1/6", .5" mesh	18	119.95

SQL>

Joining Database Tables

- ❑ Ability to combine (join) tables on common attributes is most important distinction between a relational database and other databases
- ❑ Join is performed when data are retrieved from more than one table at a time
- ❑ Join is generally composed of an equality comparison between the foreign key and the primary key of related tables
 - If you join N tables, you need N-1 join conditions
- ❑ The order of the output may be different each time you produce a listing. Use ORDER BY to have predictable results

Creating Links Through Foreign Keys

TABLE 6.11 CREATING LINKS THROUGH FOREIGN KEYS

TABLE	ATTRIBUTES TO BE SHOWN	LINKING ATTRIBUTE
PRODUCT	P_DESCRIPT, P_PRICE	V_CODE
VENDOR	V_COMPANY, V_PHONE	V_CODE

The Results of a Join

FIGURE 6.33 THE RESULTS OF A JOIN

	P_DESCRIPT	P_PRICE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE
▶	Claw hammer	\$9.95	Bryson, Inc.	Smithson	615	223-3234
	1.25-in. metal screw, 25	\$6.99	Bryson, Inc.	Smithson	615	223-3234
	2.5-in. wd. screw, 50	\$8.45	D&E Supply	Singh	615	228-3245
	7.25-in. pwr. saw blade	\$14.99	Gomez Bros.	Ortega	615	889-2546
	9.00-in. pwr. saw blade	\$17.49	Gomez Bros.	Ortega	615	889-2546
	Rat-tail file, 1/8-in. fine	\$4.99	Gomez Bros.	Ortega	615	889-2546
	Hrd. cloth, 1/4-in., 2x50	\$39.95	Randsets Ltd.	Anderson	901	678-3998
	Hrd. cloth, 1/2-in., 3x50	\$43.99	Randsets Ltd.	Anderson	901	678-3998
	B&D jigsaw, 12-in. blade	\$109.92	ORDVA, Inc.	Hakford	615	898-1234
	B&D jigsaw, 8-in. blade	\$99.87	ORDVA, Inc.	Hakford	615	898-1234
	Hicut chain saw, 16 in.	\$256.99	ORDVA, Inc.	Hakford	615	898-1234
	Power painter, 15 psi., 3-nozzle	\$109.99	Rubicon System	Orton	904	456-0092
	B&D cordless drill, 1/2-in.	\$38.95	Rubicon System	Orton	904	456-0092
	Steel matting, 4'x8'x1/8", .5" mesh	\$119.95	Rubicon System	Orton	904	456-0092

```
SELECT PRODUCT.P_DESCRIPT, PRODUCT.P_PRICE,  
VENDOR.V_NAME, VENDOR.V_CONTACT, VENDOR.V_AREACODE, VENDOR.V_PHONE  
FROM PRODUCT, VENDOR  
WHERE PRODUCT.V_CODE = VENDOR.V_CODE;  
(NOTE: table name not needed in columnlist if unambiguous)
```

An Ordered and Limited Listing After a JOIN

FIGURE 6.34 AN ORDERED AND LIMITED LISTING AFTER A JOIN

	P_DESCRIPT	P_PRICE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE
►	1.25-in. metal screw, 25	\$6.99	Bryson, Inc.	Smithson	615	223-3234
	2.5-in. wd. screw, 50	\$8.45	D&E Supply	Singh	615	228-3245
	Claw hammer	\$9.95	Bryson, Inc.	Smithson	615	223-3234
	B&D cordless drill, 1/2-in.	\$38.95	Rubicon Systems	Orton	904	456-0092
	Steel matting, 4'x8'x1/6", .5" mesh	\$119.95	Rubicon Systems	Orton	904	456-0092
	Hicut chain saw, 16 in.	\$256.99	ORDVA, Inc.	Hakford	615	898-1234

```
SELECT P_DESCRIPT, P_PRICE, V_NAME, V_CONTACT, V_AREACODE, V_PHONE  
FROM PRODUCT, VENDOR  
WHERE PRODUCT.V_CODE = VENDOR.V_CODE  
AND P_INDATE > '15-JAN-2004';
```

You can give a table an ALIAS to shorten coding

```
FROM PRODUCT P, VENDOR V  
WHERE P.V_CODE = V.V_CODE
```

MS Access uses FROM PRODUCT AS P

The Contents of the EMP Table

FIGURE 6.35 THE CONTENTS OF THE EMP TABLE

	EMP_NUM	EMP_TITLE	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_DOB	EMP_HIRE_DATE	EMP_AREACODE	EMP_PHONE	EMP_MGR
▶	100	Mr.	Kolmycz	George	D	15-Jun-42	15-Mar-85	615	324-5456	
	101	Ms.	Lewis	Rhonda	G	19-Mar-65	25-Apr-86	615	324-4472	100
	102	Mr.	Vandam	Rhett		14-Nov-58	20-Dec-90	901	675-8993	100
	103	Ms.	Jones	Anne	M	16-Oct-74	28-Aug-94	615	898-3456	100
	104	Mr.	Lange	John	P	08-Nov-71	20-Oct-94	901	504-4430	105
	105	Mr.	Williams	Robert	D	14-Mar-75	08-Nov-98	615	890-3220	
	106	Mrs.	Smith	Jeanine	K	12-Feb-68	05-Jan-89	615	324-7883	105
	107	Mr.	Diante	Jorge	D	21-Aug-74	02-Jul-94	615	890-4567	105
	108	Mr.	Wiesenbach	Paul	R	14-Feb-66	18-Nov-92	615	897-4358	
	109	Mr.	Smith	George	K	18-Jun-61	14-Apr-89	901	504-3339	108
	110	Mrs.	Genkazi	Leighla	W	19-May-70	01-Dec-90	901	569-0093	108
	111	Mr.	Washington	Rupert	E	03-Jan-66	21-Jun-93	615	890-4925	105
	112	Mr.	Johnson	Edward	E	14-May-61	01-Dec-83	615	898-4387	100
	113	Ms.	Smythe	Melanie	P	15-Sep-70	11-May-99	615	324-9006	105
	114	Ms.	Brandon	Marie	G	02-Nov-56	15-Nov-79	901	882-0845	108
	115	Mrs.	Saranda	Hermine	R	25-Jul-72	23-Apr-93	615	324-5505	105
	116	Mr.	Smith	George	A	08-Nov-65	10-Dec-88	615	890-2984	108

From this table, generate a list of employees with their manager's name

Using an Alias to Join a Table to Itself

FIGURE 6.36 USING AN ALIAS TO JOIN A TABLE TO ITSELF

	EMP_NUM	A.EMP_LNAME	EMP_MGR	B.EMP_LNAME
▶	112	Johnson	100	Kolmycz
	103	Jones	100	Kolmycz
	102	Vandam	100	Kolmycz
	101	Lewis	100	Kolmycz
	115	Saranda	105	Williams
	113	Smythe	105	Williams
	111	Washington	105	Williams
	107	Diante	105	Williams
	106	Smith	105	Williams
	104	Lange	105	Williams
	116	Smith	108	Wiesenbach
	114	Brandon	108	Wiesenbach
	110	Genkazi	108	Wiesenbach
	109	Smith	108	Wiesenbach

```
SELECT E.EMP_MGR, M.EMP_LNAME, E.EMP_NUM, E.EMP_LNAME
FROM EMP E, EMP M
WHERE E.EMP_MGR = M.EMP_NUM
ORDER BY E.EMP_MGR;
```


The Left Outer Join Results

FIGURE 6.37 THE LEFT OUTER JOIN RESULTS

```
SELECT P_CODE,  
       VENDOR.V_CODE, V_NAME  
FROM VENDOR  
LEFT JOIN PRODUCT ON  
       VENDOR.V_CODE =  
       PRODUCT.V_CODE
```

Shows all VENDOR rows and all
matching PRODUCT rows

	P_CODE	V_CODE	V_NAME
▶	23109-HB	21225	Bryson, Inc.
	SM-18277	21225	Bryson, Inc.
		21226	SuperLoo, Inc.
	SW-23116	21231	D&E Supply
	13-Q2/P2	21344	Gomez Bros.
	14-Q1/L3	21344	Gomez Bros.
	54778-2T	21344	Gomez Bros.
		22567	Dome Supply
	1546-QQ2	23119	Randsets Ltd.
	1558-QW1	23119	Randsets Ltd.
		24004	Brackman Bros.
	2232/QTY	24288	ORDVA, Inc.
	2232/QWE	24288	ORDVA, Inc.
	89-WRE-Q	24288	ORDVA, Inc.
		25443	B&K, Inc.
		25501	Damal Supplies
	11QER/31	25595	Rubicon Systems
	2238/QPD	25595	Rubicon Systems
	WR3/TT3	25595	Rubicon Systems

The Right Outer Join Results

FIGURE 6.38 THE RIGHT OUTER JOIN RESULTS

```
SELECT PRODUCT.P_CODE,  
       VENDOR.V_CODE, V_NAME  
FROM VENDOR  
RIGHT JOIN PRODUCT ON  
       VENDOR.V_CODE =  
       PRODUCT.V_CODE
```

Shows all PRODUCT rows and all
matching VENDOR rows

	P_CODE	V_CODE	V_NAME
▶	23114-AA		
	PVC23DRT		
	23109-HB	21225	Bryson, Inc.
	SM-18277	21225	Bryson, Inc.
	SW-23116	21231	D&E Supply
	13-Q2/P2	21344	Gomez Bros.
	14-Q1/L3	21344	Gomez Bros.
	54778-2T	21344	Gomez Bros.
	1546-QQ2	23119	Randsets Ltd.
	1558-QW1	23119	Randsets Ltd.
	2232/QTY	24288	ORDVA, Inc.
	2232/QWE	24288	ORDVA, Inc.
	89-WRE-Q	24288	ORDVA, Inc.
	11QER/31	25595	Rubicon Systems
	2238/QPD	25595	Rubicon Systems
	WR3/TT3	25595	Rubicon Systems